FORMAL METHODS / AUTHORIZATION KERNEL

Auths-Proof: Mechanically Linking Authorization Algebra to Shipping Rust

bordumb · bordumbb@gmail.com

27 July 2026

ABSTRACT

Authorization systems are often verified at the wrong boundary. A proof assistant may establish elegant properties of a model while production code independently reimplements the model, leaving semantic correspondence to review and testing. We present the formal core of **Auths-Proof**, a deterministic proof-carrying authorization kernel, and a refinement boundary designed to make that gap explicit and mechanically difficult to cross.

Auths-Proof models delegation as a product order over ten authority dimensions and composition as a three-valued algebra over authorized, denied, and indeterminate branch outcomes. Lean 4 proves transitivity, antisymmetry, downward-closed action coverage, strict delegation depth, threshold soundness, monotonicity, permutation invariance, termination, and linear structural cost. A versioned declarative contract generates both the shipping `no_std` Rust algebra kernel and the corresponding Lean definitions. Production authority and composition code call that generated Rust kernel. Lean exports all 1,024 Boolean attenuation projections and 2,448 threshold states through the default 16-leaf deployment bound; Rust replays those vectors against both the generated functions and the shipping composition path. Kani additionally checks the two production functions over symbolic bounded inputs.

This is not a proof of the complete verifier. The Lean theorems are unbounded, but the cross-language threshold enumeration is bounded; the contract generator, Rust compiler, cryptographic and codec layers, and the projection from rich protocol values to ten Boolean predicates remain in the trusted computing base. The result is a precise middle ground between an unlinked formal model and whole-program verification: small enough to audit, executable in production, reproducible by one command, and honest about the obligations that remain.

1. Introduction

An authorization verifier answers a deceptively small question:

> **Does authority trusted by this verifier authorize this exact action under this exact context?**

The answer is security-critical because identity is not authority. A valid signature establishes control of a key. A WebAuthn ceremony establishes control of a credential under ceremony conditions. A certificate establishes a path to a trust root. None of those facts alone grants permission to deploy software, spend a budget, invoke a tool, or mutate a repository. Authentication logics and authorization logics have long treated these as different judgments (Burrows, Abadi, and Needham 1990; Abadi et al. 1993). Capability and trust-management systems likewise make authority and delegation explicit (Dennis and Horn 1966; Blaze, Feigenbaum, and Lacy 1996; Blaze et al. 1999).

Auths-Proof implements this separation as an offline kernel:

$$\text{verify}(P, A, C) \rightarrow \text{Authorized}(S) \mid \text{Denied}(d) \mid \text{Indeterminate}(q).$$

P is a portable proof graph, A is a profile-canonical action, and C is verifier-trusted context. The kernel performs no network, clock, filesystem, database, or key-custody I/O. Only `Authorized` contains a sealed action value eligible for execution. `Denied` records a stable permanent failure. `Indeterminate` records a stable missing trustworthy fact. The latter two outcomes never permit execution.

This paper is about a narrower problem than the full protocol: how do we know that the algebra proved in Lean is the algebra executed by shipping Rust?

Writing the same function twice does not answer that question. Tests over a few examples do not answer it. A shared

trait name does not answer it. Even a machine-checked proof can create false confidence when its definitions are detached from production. Experience from verified compilers and kernels shows that useful assurance depends on an explicit refinement chain and an explicit trusted computing base (Leroy 2009; Klein et al. 2009).

Auths-Proof therefore treats linkage as a first-class artifact. A small versioned contract generates the finite algebra surface in both languages. Lean proves unbounded mathematical properties over those generated definitions. Production Rust calls the generated Rust functions. Lean emits semantic vectors; Rust consumes them. Kani analyzes the same Rust functions with symbolic inputs. Figure 1 summarizes the chain.

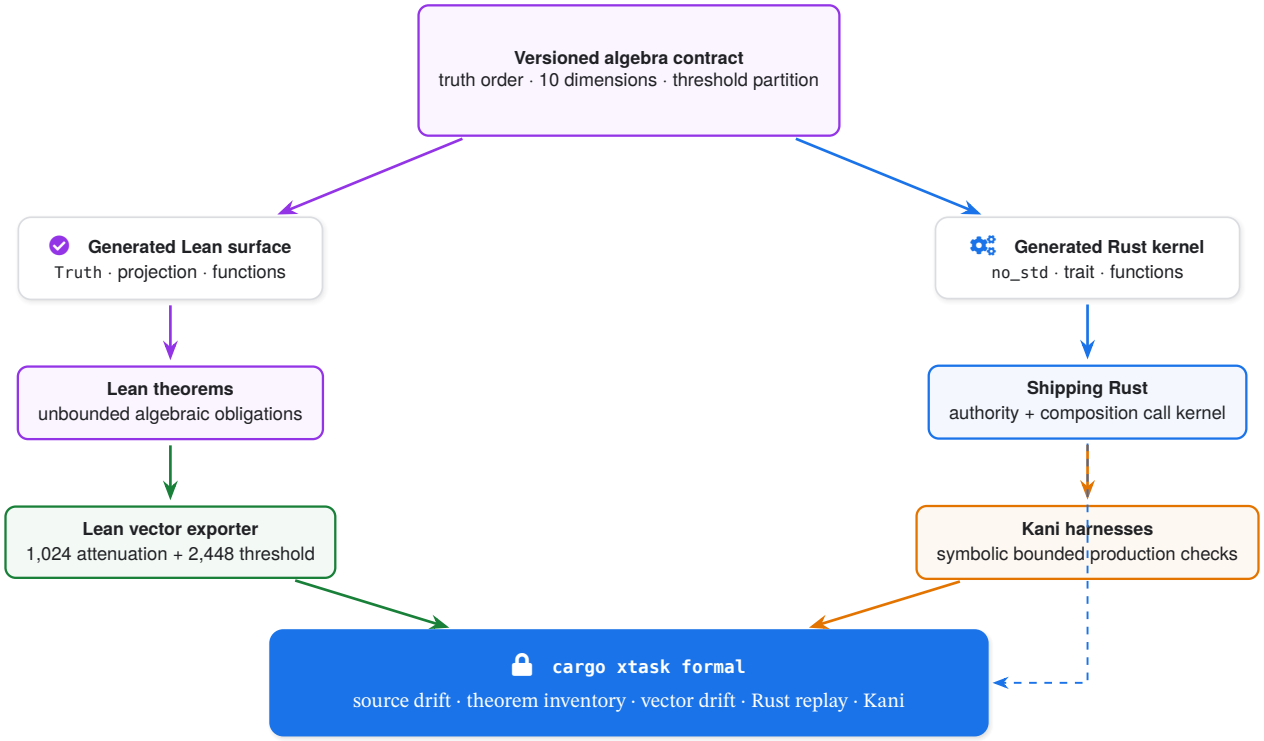

Figure 1: The mechanical refinement boundary. One contract generates the definitions used by Lean and production Rust. The proof, exhaustive finite replay, and bounded model-checking paths converge in one reproducible gate.

1.1 Contributions

This paper makes five contributions.

A closed authorization algebra. Delegation is a product order over root, depth, profile, permissions, validity, audiences, action constraint, budget, status, and assurance. Composition is a three-valued algebra that preserves trustworthy uncertainty.

An unbounded Lean model. The Lean development proves the order-theoretic, coverage, threshold, determinism, termination, and cost properties needed by the V1 algebra. The checked source contains no `sorry`, `admit`, or new axioms.

A generated language boundary. A declarative TOML contract generates the Rust trait and functions and the corresponding Lean structure and functions. Changing a dimension is a cross-language schema change rather than two independent edits.

Production refinement evidence. Shipping authority and composition code execute the generated Rust kernel. Lean-originated vectors cover the entire finite Boolean attenuation space and the entire threshold count space through the default deployment bound. Kani checks the same functions symbolically.

An explicit assurance ledger. We distinguish what is proved, what is exhaustively checked under a bound, what is tested, and what remains trusted. The artifact does not claim whole-verifier formal verification.

1.2 Scope and terminology

The paper uses *proof* in three different senses:

- an **authorization proof** is untrusted protocol input;
- a **Lean proof** is a kernel-checked theorem;
- a **Kani proof harness** is a bounded symbolic program check.

These are not interchangeable. The formal model excludes parsing, cryptography, principal adapters, graph resolution, clocks, status evidence, and complete verifier control flow. Those components are relevant to the system, but the formal claim in this paper is confined to authority attenuation and branch composition.

2. Authorization semantics

2.1 Trusted context and portable evidence

The portable proof carries signed grants, signed actions, evidence objects, bindings, and an authorization plan. It does not carry verifier trust. Trust anchors, accepted registries, evaluation time, expected challenge and audience, status and assurance policy, composition floors, and resource limits enter through C.

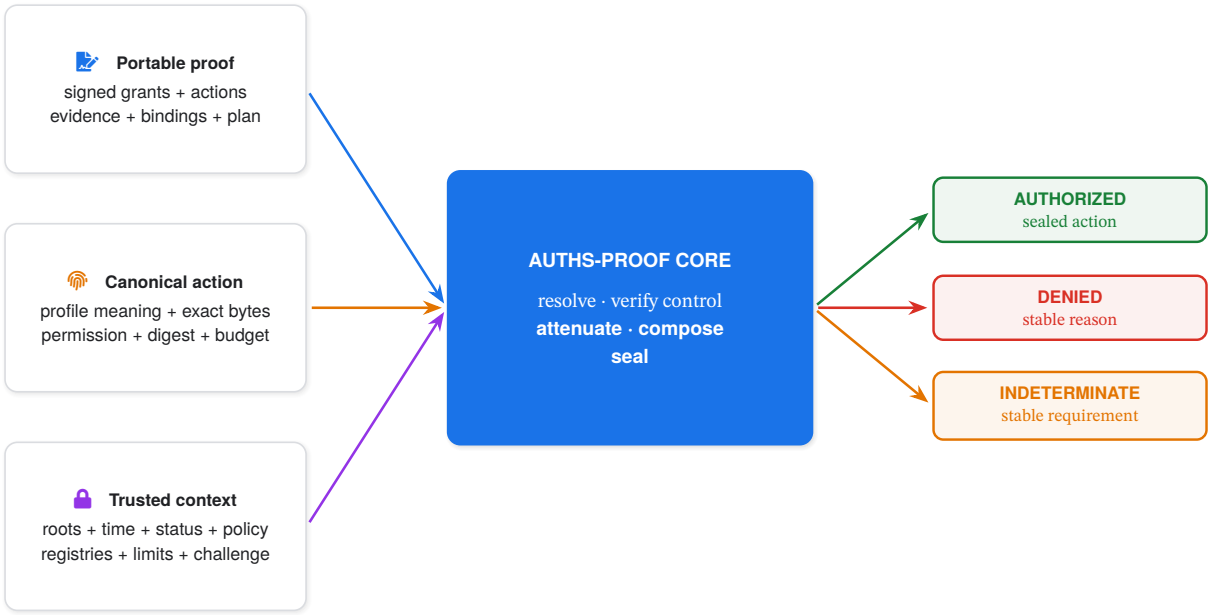

Figure 2: The verifier's narrow waist. Identity evidence contributes facts, but only the local authority computation can produce a sealed action.

This partition instantiates verifier sovereignty. A proof can demonstrate a chain from a root, but it cannot choose the local root. It can carry a status statement, but it cannot choose the accepted status method or freshness limit. It can request a composition plan, but the verifier can impose additional branch, actor, and root diversity floors.

2.2 Three-valued truth

Each proof branch produces one value in

$$\mathbb{T} = \{\bot, ?, \top\}, \qquad \bot \preceq ? \preceq \top,$$

where $\top$ is authorized, $\bot$ is denied, and $?$ is indeterminate. Indeterminate is not an error code disguised as authority. It means that a recognized trustworthy fact is absent or unavailable and that authorization would still be reachable if that fact were supplied.

For conjunction and disjunction:

$$x \wedge y = \min_{\preceq}(x, y), \qquad x \vee y = \max_{\preceq}(x, y).$$

The implementation evaluates all members in canonical order so diagnostic selection remains deterministic even when the truth algebra is permutation invariant.

For a threshold requiring k successes, let a be the number of authorized branches and u the number of indeterminate branches:

$$\text{threshold}(k, a, u) = \begin{cases} \top & a \geq k, \\ ? & a < k \wedge a + u \geq k, \\ \bot & a + u < k. \end{cases}$$

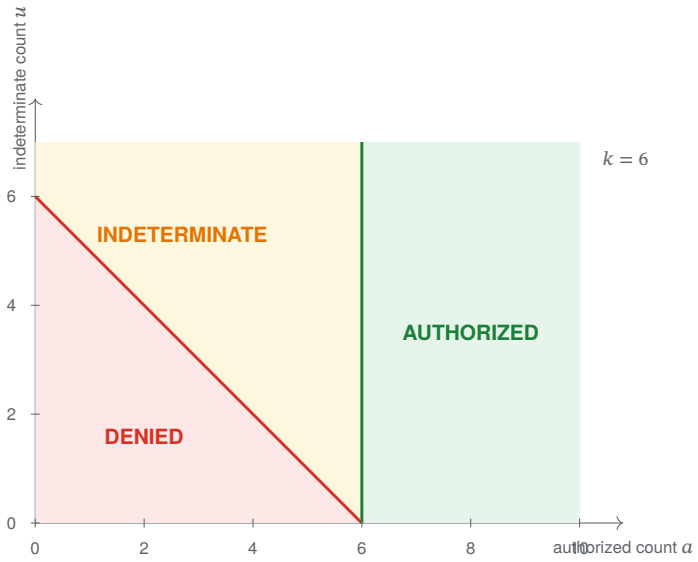

Figure 3: Threshold partition for $k = 6$. The three regions are total and mutually exclusive. Increasing k moves the authorization boundary rightward and cannot create authority.

Preserving ? matters operationally. Collapsing it into $\bot$ loses the difference between "the statement is invalid" and "a required status snapshot is unavailable." Collapsing it into $\top$ fails open.

2.3 Delegation as a product order

An effective authority value is modeled in Lean as:

$$E = (r, s, p, \pi, v, a, c, b, t, h, d),$$

where r is the root, s the current subject, p profile authority, π permissions, v validity, a audiences, c action constraint, b budget, t status, h assurance, and d remaining delegation depth. The subject changes when authority is delegated; the other coordinates form the attenuation relation.

For child E' and parent E:

$$\begin{aligned} E' \sqsubseteq E \iff\ & r' = r \wedge p' \leq p \wedge \pi' \leq \pi \wedge v' \leq v \wedge a' \leq a \\ & \wedge c' \leq c \wedge b' \leq b \wedge t' \leq t \wedge h' \leq h \wedge d' \leq d. \end{aligned}$$

A valid delegation is stricter:

$$\text{delegates}(E, E') \iff E' \sqsubseteq E \wedge d' < d.$$

The production protocol has rich domain-specific orders. Permission and audience sets use subset. Validity uses interval containment. Action constraints order `AnyBody`, allowed digest sets, and exact digests. Bounded budgets use their selected algebra. Snapshot policy preserves the method and can only reduce accepted age. Assurance policy cannot weaken.

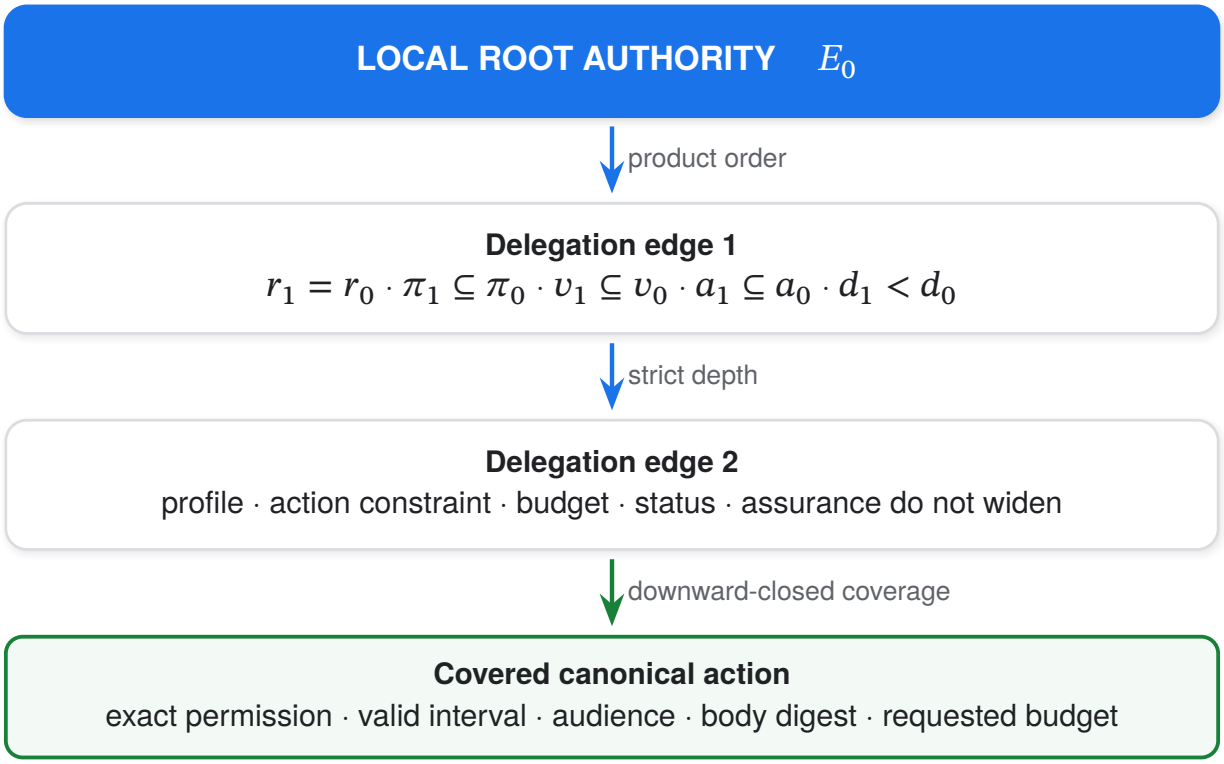

Figure 4: Authority narrows along a chain. If the terminal authority covers an action, every ancestor also covers it; the converse is intentionally false. Strictly decreasing depth bounds chain length.

3. Lean specification

3.1 Why Lean

Lean 4 combines an interactive theorem prover with an executable functional language and a small proof-checking kernel (Moura and Ullrich 2021). The Auths-Proof model uses ordinary inductive types, structures, recursive functions, and theorems. `omega` discharges Presburger arithmetic obligations; finite truth tables are proved by case analysis.

The formal project is intentionally small. It does not model Rust memory, cryptographic primitives, CBOR, or dynamic adapter behavior. Instead it defines the semantic center that is both security-critical and stable enough to merit an unbounded mathematical treatment.

The generated truth type is:

```
inductive Truth where
  | denied
  | indeterminate
  | authorized
  deriving BEq, DecidableEq, Repr
```

The generated attenuation surface is ten named Booleans. The abstract model then interprets those names as order relations over natural-number coordinates. This separation lets Lean prove order properties while the generated surface remains isomorphic to production's finite decision boundary.

3.2 Attenuation theorems

The attenuation development proves:

1. reflexivity and transitivity of $\sqsubseteq$;
2. antisymmetry when subjects agree;
3. downward-closed action coverage;
4. root preservation and strict depth for delegation;

5. transitive non-widening across delegation chains;
6. coverage of an authorized child action by its parent;
7. equivalence between the generated Boolean kernel and abstract delegation.

The central refinement theorem is executable documentation:

```
theorem attenuation_kernel_refines
    (parent child : EffectiveAuthority) :
    Generated.attenuationAccepts
        (delegationProjection parent child) = true
      ↔ delegates parent child := by
  simp [Generated.attenuationAccepts,
        delegationProjection, delegates, attenuates]
  omega
```

This theorem is stronger than separately proving the conjunction function and the abstract order. It states that the generated acceptance result is true exactly when the abstract delegation judgment holds.

The theorem `coverage_downward_closed` establishes:

$$E' \sqsubseteq E \wedge \mathrm{covers}(E', A) \Longrightarrow \mathrm{covers}(E, A).$$

This is the safety direction required for attenuation. Delegating narrower authority cannot make the descendant authorize an action that the ancestor could not authorize.

3.3 Composition theorems

For both `all` and `any`, Lean proves commutativity, associativity, and idempotence. It proves that one-of-two is `any`, two-of-two is `all`, and that tightening a threshold cannot increase truth:

$$\mathrm{thresholdTwo}(2, x, y) \preceq \mathrm{thresholdTwo}(1, x, y).$$

The unbounded count classifier has three soundness theorems:

$$\begin{aligned}
\mathrm{threshold}(k, a, u) = \top &\Rightarrow k \leq a, \\
\mathrm{threshold}(k, a, u) = \bot &\Rightarrow a + u < k, \\
\mathrm{threshold}(k, a, u) = ? &\Rightarrow a < k \leq a + u.
\end{aligned}$$

Plan traversal is modeled structurally. Lean proves that the defined visit list is exactly the leaf list, that every finite plan has a node count, and that the declared structural cost equals that count. These are algebra-level results, not a cost proof for cryptography or allocation.

Composition floors are ordered componentwise over authorized branches, distinct actors, and distinct roots. The model proves that a tighter satisfied floor implies every looser floor is also satisfied. Thus raising local diversity requirements cannot create authorization.

3.4 Proof inventory

Table 1 groups the checked obligations. The formal source currently contains 32 named theorems; the release manifest inventories 29 public algebra/refinement obligations, while three local helper/diversity declarations remain outside that release list.

Family	Representative obligations	Count
Attenuation	reflexive, transitive, antisymmetric, coverage closure, root, depth, chain attenuation, generated-kernel refinement	12
Composition	semilattice laws, threshold identities and monotonicity, permutation-stable truth and diagnostics, plan traversal, termination, structural cost, three threshold soundness directions	17
Helpers and diversity	truth-rank injectivity, canonical-code commutativity, tighter-floor monotonicity	3
Lean source total		**32**

Table 1: Lean theorem inventory. Counts describe named declarations, not proof difficulty or whole-system coverage.

4. From model to shipping code

4.1 The correspondence problem

A traditional refinement approach can prove that an implementation simulates an abstract specification. CompCert and seL4 demonstrate the strength of such end-to-end refinement arguments (Leroy 2009; Klein et al. 2009). Translation validation instead validates the result of a particular translation rather than proving a translator correct for all inputs (Pnueli, Siegel, and Singerman 1998).

Auths-Proof currently occupies a smaller point in this design space. It does not extract the full Rust verifier into Lean, and it does not verify the contract generator. It generates the small common algebra surface, proves the Lean interpretation, executes the generated Rust surface in production, and checks finite semantic agreement.

Tools such as Aeneas translate safe Rust into functional models for proof assistants (Ho and Protzenko 2022), while Verus verifies rich properties directly over Rust-like programs (Lattuada et al. 2024). Those approaches are promising future routes for reducing the remaining projection and generator trust. The present design was chosen because its boundary is smaller than the toolchain required to translate the complete verifier.

4.2 One declarative contract

The source of the shared surface is `formal/algebra-contract-v1.toml`. Its essential shape is:

```toml
schema = "auths-proof-algebra-contract/v1"
exhaustive_threshold_bound = 16
truth_order = ["denied", "indeterminate", "authorized"]
attenuation_acceptance = "all"

[[attenuation_dimensions]]
rust = "root_preserved"
lean = "rootPreserved"

[[attenuation_dimensions]]
rust = "depth_decreases"
lean = "depthDecreases"

[threshold]
authorized = "authorized >= required"
indeterminate =
  "authorized < required &&
   authorized + indeterminate >= required"
denied = "authorized + indeterminate < required"
```

The real contract lists all ten dimensions. The generator parses a closed typed TOML schema, rejects unknown fields, checks unique names, and accepts only the declared V1 truth order and formulas. It then deterministically renders:

- `core/crates/auths-algebra-kernel/src/generated.rs`;

- `formal/Auths/Generated/Algebra.lean`.

Normal verification compares both files byte-for-byte with fresh renderings. Intentional changes require `cargo xtask formal --update`.

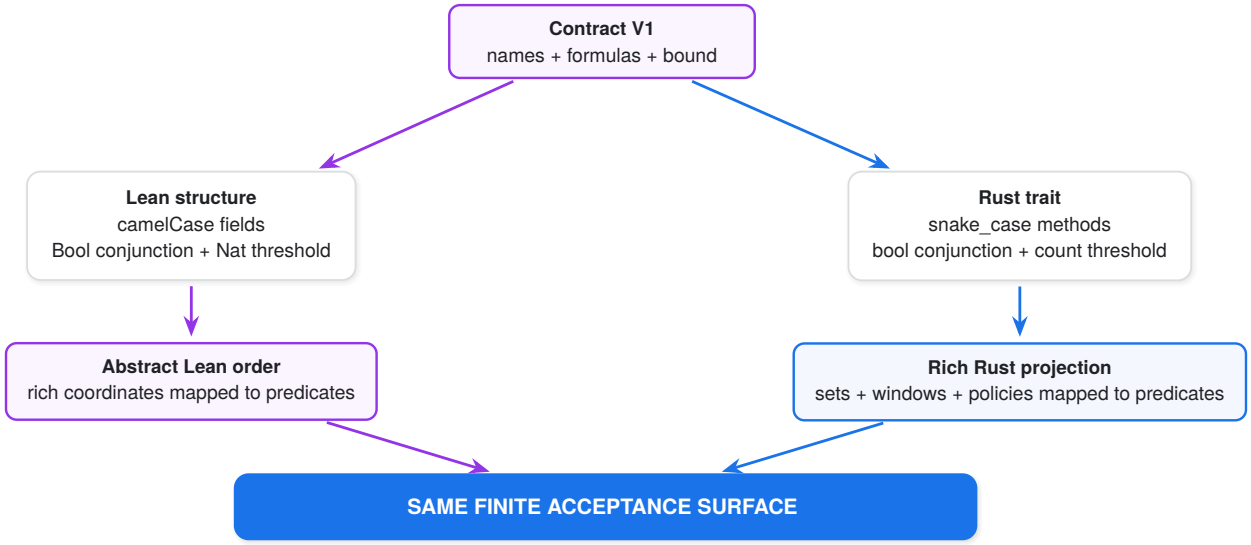

Figure 5: Generated structural correspondence. The contract prevents silent field drift. Semantic correctness of each rich Rust predicate remains a separate obligation.

4.3 Shared traits, correctly understood

There is no runtime trait object shared between Lean and Rust. Instead, the contract generates structurally corresponding interfaces:

```rust
pub trait AttenuationProjection {
    fn root_preserved(&self) -> bool;
    fn depth_decreases(&self) -> bool;
    // ... eight more named dimensions
}
```

```
structure AttenuationProjection where
  rootPreserved : Bool
  depthDecreases : Bool
  -- ... eight more named dimensions
```

This is stronger than manually duplicating traits because one source controls the field set, ordering, names, and aggregation semantics. It is weaker than proving a compiler from Rust to Lean. The distinction is central to the paper's claim.

Adding an eleventh attenuation dimension causes deliberate breakage:

1. the generated Rust trait changes;
2. the generated Lean structure changes;
3. Rust projection implementations fail to compile until they supply it;
4. Lean projections fail to elaborate until they supply it;
5. vector cardinality and exporter logic must change;
6. the checked-in generated artifacts drift until explicitly regenerated.

That failure mode turns semantic expansion into an auditable repository-wide event.

4.4 Production Rust routing

The generated kernel is a separate `no_std`, `unsafe`-free crate. Shipping composition no longer owns a handwritten threshold classifier:

```rust
pub fn evaluate_threshold_counts(
    k: u16,
    authorized: usize,
    indeterminate: usize,
) -> ThresholdTruth {
    auths_algebra_kernel::threshold_counts(
        k, authorized, indeterminate
    )
}
```

Shipping authority constructs a `GrantAttenuation` from rich domain checks and passes it to `attenuation_accepts`. The generated function accepts exactly the conjunction of all trait methods.

Root preservation deserves special attention. In Rust, root is not supplied by an untrusted child grant; `EffectiveAuthority::delegate` mutates the existing state while retaining its private root field. The production projection therefore returns `true` for `root_preserved`. Lean models root explicitly and proves that an accepted abstract delegation preserves it. This is a legitimate representation difference, but it belongs in the trusted mapping rather than being hidden.

5. Executable refinement evidence

5.1 Lean-generated vectors

The Lean executable `Auths.VectorExport` is the semantic vector producer. It does not call a Rust reference evaluator. For attenuation, it enumerates all assignments in $\{0,1\}^{10}$:

$$2^{10} = 1{,}024 \text{ cases.}$$

Exactly one vector, the all-true assignment, is accepted because V1 aggregation is conjunction.

For threshold counts, the declared exhaustive bound is $B = 16$. The exporter enumerates:

$$1 \le k \le B, \quad 0 \le a \le B, \quad 0 \le u \le B - a.$$

The case count is:

$$B \sum_{a=0}^{B} (B - a + 1) = 16 \cdot 153 = 2{,}448.$$

Rust checks every vector against `auths_algebra_kernel::threshold_counts`. It also constructs a real 16-leaf `AuthorizationPlan::k_of_n`, feeds the declared mix of branch outcomes to shipping `auths_composition::evaluate`, and checks that the result and full leaf visitation agree.

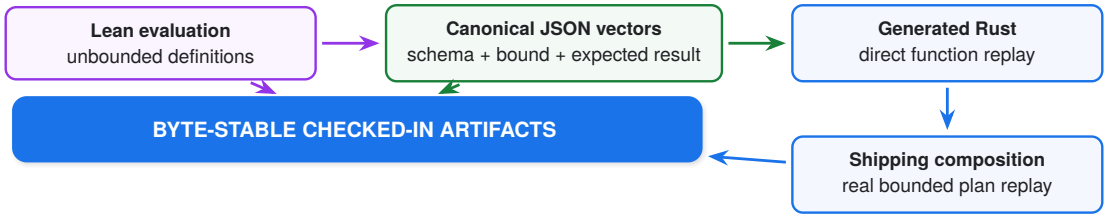

Figure 6: Semantic vector provenance. Expected values originate in Lean. Rust is a consumer, not a co-author of the oracle.

5.2 Kani over the shipping kernel

Kani lowers Rust MIR into CBMC's bit-precise model-checking pipeline and checks assertions over symbolic inputs (Delmas et al. 2026; Kroening, Schrammel, and Tautschnig 2023). Two harnesses target the generated production crate.

The threshold harness selects symbolic `u16` values, assumes the declared default bound and a valid positive threshold, calls `threshold_counts`, and asserts the three-way partition. Kani also checks arithmetic overflow and runtime safety on reachable paths.

The attenuation harness selects ten symbolic Booleans and asserts:

```
attenuation_accepts(&checks)
    == checks.root_preserved
    && checks.depth_decreases
    && checks.profile_attenuates
    && checks.permissions_attenuate
    && checks.validity_attenuates
    && checks.audiences_attenuate
    && checks.action_constraint_attenuates
    && checks.budget_attenuates
    && checks.status_attenuates
    && checks.assurance_attenuates
```

Kani contributes implementation-level evidence: the actual compiled Rust functions satisfy these assertions over the bounded harness domain. It does not replace the unbounded Lean theorems, and the Lean theorems do not replace Kani's check of Rust arithmetic and control flow.

5.3 One reproducible gate

`cargo xtask formal` performs, in order:

1. parse and validate the contract;
2. render both generated modules and reject byte drift;
3. build the Lean project;
4. execute the Lean vector exporter and reject vector drift;
5. scan selected formal source for `sorry`, `admit`, and `axiom`;
6. verify every release-inventory theorem name is declared;
7. run Rust refinement tests over all vectors;
8. run both Kani harnesses.

The update mode is explicit:

```
cargo xtask formal --update
```

It regenerates both language modules and both vector files. A normal run never silently updates proof evidence.

5.4 Evaluation results

Artifact	Observed result	Established scope
Lean source	32 named theorems; 29 release-inventoried	unbounded abstract attenuation, composition, traversal, and diversity facts
Attenuation vectors	1,024 / 1,024 replayed	all Boolean assignments over ten generated predicates
Threshold vectors	2,448 / 2,448 replayed	all valid count states through the default 16-leaf bound
Shipping plan replay	2,448 / 2,448 agreed	real ‘k-of-n‘ evaluation agrees with the Lean oracle at that bound
Kani	2 / 2 harnesses; 0 failures	symbolic bounded generated Rust functions and runtime checks
Generated drift	no drift	checked-in Rust, Lean, and vectors match deterministic generation

Table 2: Formal artifact results at the evaluated revision. These are semantic checks, not performance measurements.

6. Assurance boundary

6.1 What is proved

The Lean kernel checks the following model-level claims:

- attenuation is a partial order modulo subject identity;
- delegation preserves the root, never widens authority, and strictly reduces depth;
- terminal action coverage implies ancestor coverage;
- `all` and `any` have the expected semilattice laws;
- threshold results imply their defining count inequalities;
- tightening a two-branch threshold cannot increase truth;
- truth and canonical diagnostic selection are permutation invariant;
- finite plans terminate under the structural model and have linear declared node cost;
- tighter diversity floors cannot create authorization;
- generated attenuation acceptance is equivalent to abstract delegation.

These proofs quantify over natural numbers and finite inductive plans. They are not limited to 16 leaves.

6.2 What is exhaustively checked under a bound

The cross-language threshold relation is exhaustive only through 16 leaves, which equals `DEFAULT_MAX_PLAN_LEAVES` for target V1. The protocol model also contains a separately configurable hard maximum of 128. The artifact does not enumerate or model-check every threshold count through 128.

The attenuation projection space is genuinely complete for the generated Boolean interface because it has exactly ten inputs. That completeness does not prove that each rich Rust predicate computes the intended domain relation.

6.3 What remains trusted

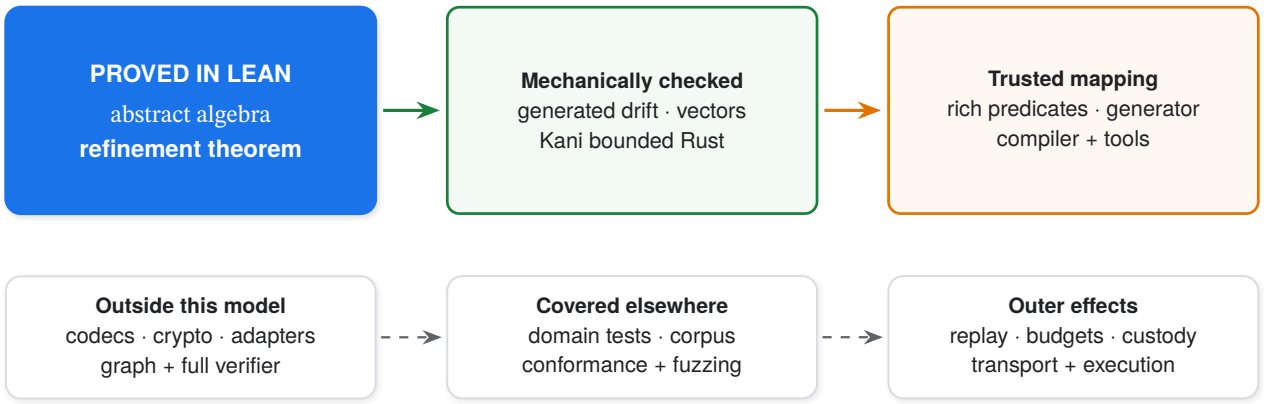

Figure 7: Claim and trust boundary. Dark blue is theorem-proved; green is mechanically cross-checked; amber remains trusted. The lower row is outside the formal model.

The trusted computing base includes:

- the Lean kernel and imported tactics;
- the deterministic contract generator;
- the Rust compiler and Kani/CBMC toolchain;
- the mapping from rich Rust values to ten Boolean predicates;
- the model's adequacy as a specification of intended authorization;
- everything outside the algebra, including codecs and cryptography.

Rust's type system and `unsafe`-free core reduce memory-safety risk, but Rust language safety is not itself a proof of functional authorization correctness. RustBelt provides a machine-checked foundation for important Rust safety claims and illustrates why language safety and application correctness must remain separate statements (Jung et al. 2018).

6.4 The projection gap

The largest remaining semantic gap is not the ten-input conjunction. It is the construction of those inputs:

```
GrantAttenuation {
    permissions_attenuate:
        grant.permissions().is_subset_of(&self.permissions),
    validity_attenuates:
        self.validity.contains_window(grant.validity()),
    action_constraint_attenuates:
        grant.action_constraint().attenuates(
            &self.action_constraint
        ),
    // profile, audience, budget, status, assurance, depth
}
```

Each method has ordinary Rust tests, but the Lean model abstracts it to an ordered natural-number coordinate. A future refinement should either:

1. translate this isolated safe-Rust projection into Lean with Aeneas;
2. model each rich domain type and prove its Rust relation against generated vectors;
3. use a Rust-native deductive verifier such as Verus for the projection;
4. combine these approaches for defense in depth.

Whole-verifier translation is not the immediate next step. The highest-value work is to close this narrow projection gap first.

7. Security consequences

7.1 Delegation escalation

The product-order design makes widening explicit. A child must not:

- change the local root;
- keep or increase remaining depth;
- select a broader or different profile;
- add permissions or audiences;
- extend validity;
- broaden action-body discretion;
- remove or increase a bounded budget;
- weaken status freshness;
- weaken assurance.

The aggregate function cannot accidentally omit a declared dimension because the generated trait and generated conjunction share the contract's field list. The projection can still miscompute a dimension; that is the residual gap described above.

7.2 Fail-closed uncertainty

Three-valued threshold semantics prevent unavailable evidence from being treated as success. They also preserve enough information to distinguish a permanent denial from a potentially satisfiable missing fact. The threshold soundness theorems show:

- authorization requires at least k established branches;
- denial means even every indeterminate branch cannot reach k;
- indeterminate means k is not established but remains reachable.

No diagnostic ordering can change that truth result. Canonical reason selection exists for deterministic reporting, not as an input to authorization.

7.3 Bounded work

Strictly decreasing depth gives a simple termination measure for delegation chains. Structural plan recursion terminates because plans are finite inductive values. Production additionally validates deployment limits before evaluation and reserves work before variable-cost adapters and cryptography.

The Lean theorem `evaluation_cost_linear_in_nodes` concerns the declared structural cost function. It should not be read as an empirical runtime complexity proof for the full verifier. Allocation, hashing, signature verification, adapter work, and codec behavior remain outside that theorem.

7.4 Change control as a security property

Authorization semantics evolve. The generated boundary is therefore valuable even if no current theorem fails. It changes the review shape:

> **SEMANTIC CHANGE RULE**
>
> A new truth value, threshold rule, or attenuation dimension must appear as one contract change whose generated Rust, generated Lean, proofs, vector cardinality, production projection, and checked artifacts change together.

This is not merely build convenience. It prevents a class of silent specification drift in which a production check is added without a formal counterpart, or a formal obligation is strengthened without changing shipping behavior.

8. What the core enables

The verified algebra is deliberately below transport and product policy. That placement lets outer packages vary without redefining authority.

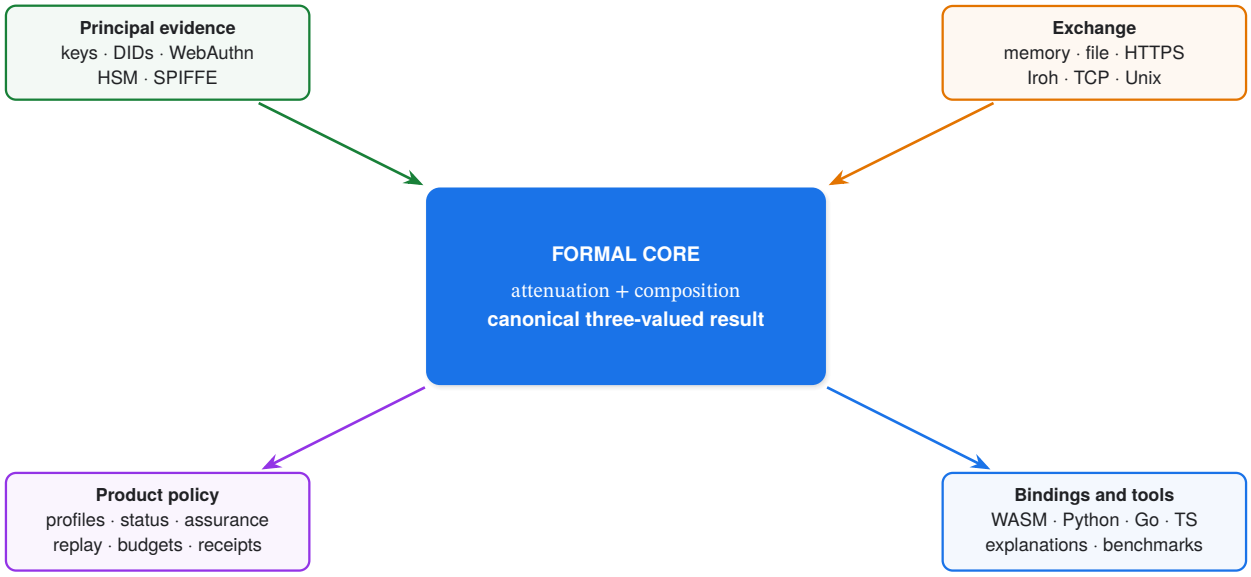

Figure 8: The algebra as a narrow semantic center. Outer components supply evidence, carry bytes, enforce state, and expose APIs; they do not redefine attenuation or composition.

Seven principal methods can produce typed control evidence for the same verifier. Exchange transports carry opaque proof bytes but cannot upgrade an invalid proof. Product packages can add atomic replay and budget stores, receipts, custody, profiles, and enforcement. Native and WASM boundaries can consume the same canonical result. Deployment explanations can report which trusted-context facts caused or prevented authorization.

These packages demonstrate architectural leverage, not additional formal coverage. A correct algebra cannot compensate for an adapter that overstates control, a profile that assigns unsafe meaning, a stale status source, or an executor that ignores the sealed action.

9. Related work

9.1 Authorization and attenuation

Capability systems made authority an explicit transferable object (Dennis and Horn 1966). SPKI authorization certificates bind permissions to keys and support delegation (Ellison et al. 1999). SDSI emphasizes linked local namespaces (Rivest and Lampson 1996). Macaroons support efficient contextual attenuation through caveats (Birgisson et al. 2014). Trust-management systems such as PolicyMaker, KeyNote, and RT separate assertions, local policy, and compliance checking (Blaze, Feigenbaum, and Lacy 1996; Blaze et al. 1999; Li, Mitchell, and Winsborough 2002).

Auths-Proof shares explicit delegation and local policy but uses a closed multidimensional order rather than a general policy language. That restriction makes the core algebra finite at its production projection boundary and tractable for exhaustive cross-language checking.

Proof-carrying code shifts the burden of supplying safety evidence to the producer (Necula 1997). Proof-carrying authentication applies a related idea to distributed authorization judgments (Appel and Felten 1999; Bauer, Schneider, and Felten 2002). Auths-Proof authorization proofs are protocol evidence, not Lean proof terms; the receiver still executes a fixed verifier.

9.2 Formal refinement

CompCert proves semantic preservation through a realistic compiler (Leroy 2009). seL4 proves refinement from an abstract specification to a C implementation (Klein et al. 2009). Translation validation checks a particular translation result (Pnueli, Siegel, and Singerman 1998). Auths-Proof's generated boundary is closer in spirit to a small translation-validation artifact, but its generator is not verified and the rich projection is not yet translated.

Lean 4 supplies the theorem-proving environment (Moura and Ullrich 2021). Aeneas uses functional translation to make safe Rust amenable to proof assistants (Ho and Protzenko 2022). Verus provides a practical Rust-centered systems-verification environment (Lattuada et al. 2024). These systems define credible paths toward a stronger im-

plementation refinement than the current generated correspondence.

9.3 Model checking and testing

Kani and CBMC provide bit-precise bounded model checking over the shipping Rust function (Delmas et al. 2026; Kroening, Schrammel, and Tautschnig 2023). This complements, rather than duplicates, Lean: Kani sees Rust control flow and machine arithmetic, while Lean proves unbounded properties of the mathematical definition.

Property-based testing and fuzzing remain valuable at the richer boundaries that the Lean model excludes (Claessen and Hughes 2000; Miller, Fredriksen, and So 1990). They search large input spaces and preserve counterexamples, but passing campaigns do not establish universal properties. The artifact therefore labels theorem, exhaustive enumeration, bounded model checking, conformance testing, and fuzzing separately.

10. Limitations and future work

> **This paper does not claim full functional correctness of the verifier.** It establishes an unbounded abstract algebra, a generated structural correspondence, exhaustive finite agreement for declared domains, and bounded checks of two shipping Rust functions.

The most important limitations are:

Model adequacy. A correct proof of the wrong model is still wrong. Review of the chosen authority dimensions and their order remains essential.

Rich projection. Permission sets, intervals, action constraints, budgets, status, and assurance are reduced to Booleans by unverified Rust relations.

Generator trust. The generator is deterministic and drift-checked but not proved semantics-preserving.

Bound asymmetry. Lean threshold theorems are unbounded. Rust vector replay and the Kani threshold harness stop at 16, not the configurable hard maximum of 128.

Incomplete release inventory. Three named Lean helper/diversity theorems are not currently listed in the 29-entry release manifest. The source builds them, but the inventory checker does not require their names.

Whole-verifier exclusion. Parsing, graph resolution, signature verification, evidence adapters, status, assurance matching, and final sealing are tested but not refined to Lean.

Toolchain trust. Lean, Rust, Kani, CBMC, the build environment, and their dependencies remain part of the evidence pipeline.

The next work should proceed in decreasing order of semantic leverage:

1. model the rich authority types and close the Rust projection gap;
2. include every public theorem in a generated release inventory;
3. translate the isolated safe-Rust algebra/projection with Aeneas or verify it with Verus;
4. extend symbolic threshold checks to the hard bound or prove a Rust function contract independent of enumeration;
5. connect canonical codec and graph-state invariants to the formal model;
6. produce proof-carrying build attestations for generated artifacts.

11. Conclusion

Formalization is valuable only to the extent that its relationship to shipping behavior is understood. Auths-Proof does not solve that relationship by assertion. It makes the authorization algebra small, generates the finite language boundary from one contract, routes production Rust through that boundary, proves the abstract properties in Lean, exports expected behavior from Lean, replays every declared finite state in Rust, and model-checks the same Rust functions with Kani.

The resulting claim is intentionally precise:

Auths-Proof's generated attenuation and threshold kernels agree with their Lean definitions over the declared finite boundary, while Lean proves the central algebraic properties without that bound.

That claim is narrower than whole-program verification and substantially stronger than parallel handwritten models. More importantly, it exposes the remaining work: rich-domain projection, generator correctness, codecs, cryptography, adapters, and complete verifier control flow. The system can now improve along a visible refinement ladder rather than accumulating informal confidence around an isolated proof artifact.

References

Abadi, Martín, Michael Burrows, Butler Lampson, and Gordon Plotkin. 1993. "A Calculus for Access Control in Distributed Systems." *ACM Transactions on Programming Languages and Systems* 15 (4): 706–34. https://doi.org/10.1145/155183.155225.

Appel, Andrew W., and Edward W. Felten. 1999. "Proof-Carrying Authentication." In *Proceedings of the 6th ACM Conference on Computer and Communications Security*, 52–62. https://doi.org/10.1145/319709.319718.

Bauer, Lujo, Michael A. Schneider, and Edward W. Felten. 2002. "A General and Flexible Access-Control System for the Web." In *Proceedings of the 11th USENIX Security Symposium*, 93–108. USENIX Association. https://www.usenix.org/legacy/events/sec02/full_papers/bauer/bauer_html/.

Birgisson, Arnar, Joe Gibbs Politz, Úlfar Erlingsson, Ankur Taly, Michael Vrable, and Mark Lentczner. 2014. "Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud." In *Network and Distributed System Security Symposium*. Internet Society. https://research.google/pubs/macaroons-cookies-with-contextual-caveats-for-decentralized-authorization-in-the-cloud/.

Blaze, Matt, Joan Feigenbaum, John Ioannidis, and Angelos D. Keromytis. 1999. "The KeyNote Trust-Management System Version 2." RFC 2704. RFC Editor. https://doi.org/10.17487/RFC2704.

Blaze, Matt, Joan Feigenbaum, and Jack Lacy. 1996. "Decentralized Trust Management." In *Proceedings of the 1996 IEEE Symposium on Security and Privacy*, 164–73. https://doi.org/10.1109/SECPRI.1996.502679.

Burrows, Michael, Martín Abadi, and Roger Needham. 1990. "A Logic of Authentication." *ACM Transactions on Computer Systems* 8 (1): 18–36. https://doi.org/10.1145/77648.77649.

Claessen, Koen, and John Hughes. 2000. "QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs." In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming*, 268–79. https://doi.org/10.1145/351240.351266.

Delmas, Rémi, Zyad Hassan, Qinheping Hu, Rahul Kumar, Felipe R. Monteiro, Thanh Nguyen, Adrián Palacios, et al. 2026. "Kani: A Model Checker for Rust." *arXiv Preprint arXiv:2607.01504*. https://doi.org/10.48550/arXiv.2607.01504.

Dennis, Jack B., and Earl C. Van Horn. 1966. "Programming Semantics for Multiprogrammed Computations." *Communications of the ACM* 9 (3): 143–55. https://doi.org/10.1145/365230.365252.

Ellison, Carl M., Bill Frantz, Butler Lampson, Ron Rivest, Brian M. Thomas, and Tatu Ylonen. 1999. "SPKI Certificate Theory." RFC 2693. RFC Editor. https://doi.org/10.17487/RFC2693.

Ho, Son, and Jonathan Protzenko. 2022. "Aeneas: Rust Verification by Functional Translation." *Proceedings of the ACM on Programming Languages* 6 (ICFP): 711–41. https://doi.org/10.1145/3547647.

Jung, Ralf, Jacques-Henri Jourdan, Robbert Krebbers, and Derek Dreyer. 2018. "RustBelt: Securing the Foundations of the Rust Programming Language." *Proceedings of the ACM on Programming Languages* 2 (POPL): 66:1–34. https://doi.org/10.1145/3158154.

Klein, Gerwin, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, et al. 2009. "seL4: Formal Verification of an OS Kernel." In *Proceedings of the 22nd ACM Symposium on Operating Systems Principles*, 207–20. https://doi.org/10.1145/1629575.1629596.

Kroening, Daniel, Peter Schrammel, and Michael Tautschnig. 2023. "CBMC: The C Bounded Model Checker." *arXiv Preprint arXiv:2302.02384*. https://doi.org/10.48550/arXiv.2302.02384.

Lattuada, Andrea, Travis Hance, Jay Bosamiya, Matthias Brun, Chanhee Cho, Hayley LeBlanc, Pranav Srinivasan, et al. 2024. "Verus: A Practical Foundation for Systems Verification." In *Proceedings of the 30th ACM Symposium on Operating Systems Principles*, 438–54. https://doi.org/10.1145/3694715.3695952.

Leroy, Xavier. 2009. "Formal Verification of a Realistic Compiler." *Communications of the ACM* 52 (7): 107–15. https://doi.org/10.1145/1538788.1538814.

Li, Ninghui, John C. Mitchell, and William H. Winsborough. 2002. "Design of a Role-Based Trust-Management Framework." In *Proceedings of the 2002 IEEE Symposium on Security and Privacy*, 114–30. https://doi.org/10.1109/SECPRI.2002.1004376.

Miller, Barton P., Lars Fredriksen, and Bryan So. 1990. "An Empirical Study of the Reliability of UNIX Utilities." *Communications of the ACM* 33 (12): 32–44. https://doi.org/10.1145/96267.96279.

Moura, Leonardo de, and Sebastian Ullrich. 2021. "The Lean 4 Theorem Prover and Programming Language." In *Automated Deduction – CADE 28*, 12699:625–35. Lecture Notes in Computer Science. Springer. https://doi.org/10.1007/978-3-030-79876-5_37.

Necula, George C. 1997. "Proof-Carrying Code." In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 106–19. https://doi.org/10.1145/263699.263712.

Pnueli, Amir, Michael Siegel, and Eli Singerman. 1998. "Translation Validation." In *Tools and Algorithms for the Construction and Analysis of Systems*, 1384:151–66. Lecture Notes in Computer Science. Springer. https://doi.org/10.1007/BFb0054170.

Rivest, Ronald L., and Butler Lampson. 1996. "SDSI: A Simple Distributed Security Infrastructure." Technical memo. Massachusetts Institute of Technology. https://www.microsoft.com/en-us/research/publication/sdsi-a-simple-distributed-security-infrastructure/.